Introduction of Queue

A queue is a non-primitive linear data structure commonly used in programming. It is a homogenous collection of where insertion and deletion operations are performed at two different ends. Insertion is done at the **rear (end)** of the queue, while deletion is performed from the **front (beginning)** of the queue.

Queues operate on the principle of **FIFO (First-In, First-Out)** or **LILO (Last-In, Last-Out)**. This means the element inserted first is removed first.

A queue uses two variables:

- **Front** → points to the first element
- **Rear** → points to the last inserted element

Examples: line of people waiting for tickets, queue at a bank, printer queue, vehicles at a traffic signal, etc.

Queue Related Terms

- **Rear:** A variable indicating the end where new data will be added in the queue.
- **Front:** It is a variable indicating the in from where data will be retrieved.
- The end from which items are deleted is called front of the queue
- Since first Inserted item is removed at fist and last inserted item is remove at last. That is why queue is called FIFO (First In First Out) data structure.
- Insertion process is called **<u>enqueue</u>** operation. When we insert an element in the queue, the value of rear is increment by one: Rear = Rear + 1
- Deletion process is called **<u>dequeue</u>** operation. When we delete or remove an element from the queue, the value of the front or head is increment by one: Front = Front + 1
- The process of adding an element into the queue is called **<u>enqueue</u>** and the process of removal of an element from queue is called **<u>dequeue</u>**.

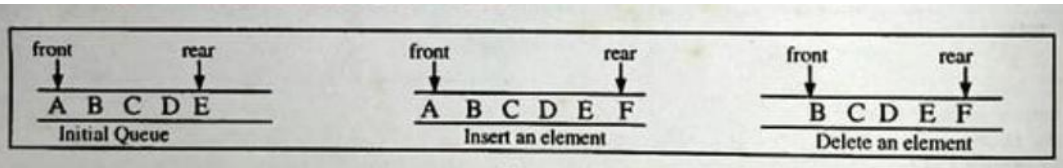

Figure 4.5

Queue as an ADT/ Types of Queue Operations

There are several operations that can be performed on a queue. They are:

1. **enqueue( ):** When a new element is inserted into the queue, such an operation is called an enqueue( ) operation. In a queue, insertion always takes place at the rear (end).
   In array implementation, while inserting a new element, the rear is incremented by 1, and the element is placed at that position. Before inserting, we must check whether the queue is full. If the queue is already full and we try to insert a new element, then queue overflow occurs, and the enqueue operation will not proceed further.
2. **dequeue( )**: When an element is removed from the queue, such an operation is called a dequeue( ) operation. In a queue, deletion always occurs from the front (beginning).
   While deleting an element, the front is incremented by 1, pointing to the next element in the queue. Similar to stack, the element is not physically removed but is logically deleted and no longer accessible.
   If we try to perform dequeue on an empty queue, it results in queue underflow.
3. **peek( ) / front( )**: The peek( ) or front( ) operation returns the element at the front of the queue, i.e., the element that was inserted first. It only displays the element without removing it from the queue.
4. **display( )**: The display( ) operation prints or shows all the elements of the queue from front to rear, in the order in which they were inserted.
5. **is**Em**pty( ) / Underflow:** The isEmpty( ) operation is used to check whether the queue contains any elements or not. It returns true (1) if the queue is empty, otherwise false (0). The main purpose of this operation is to prevent queue underflow, which occurs when attempting operations like dequeue( ) or peek( ) on an empty queue.
6. isFull( ) / Overflow: The isFull( ) operation checks whether the queue has reached its maximum capacity. Since array-based queues have a fixed size, they cannot grow dynamically. This operation helps prevent queue overflow, which occurs when trying to insert (enqueue) an element into a queue that is already full. It returns true if the queue is full, otherwise false.

Implementation of Queue

A queue can be implemented in the following two ways:

1. Array-Based Implementation of Queue (Static Implementation)
2. Linked List-Based Implementation of Queue (Dynamic Implementation)

Array-Based Implementation of Queue
To implement a queue, we use the concept of an array. The properties of the array become the properties of the queue as well. In an array-based queue, a fixed size is defined.
Two index pointers are used:

- front → points to the first element
- rear → points to the last element

Initially, front = -1 and rear = -1, indicating that the queue is empty. When the first element is inserted, both front and rear are set to 0.

<u>Algorithm for enqueue( ) operation</u>

```
enqueue(item)
  if rear == MAX_SIZE - 1
    print "Queue Overflow"
  else
    if front == -1
      set front ← 0
    set rear ← rear + 1
    set queue[rear] ← item
```

<u>Algorithm for peek( ) / front( ) operation</u>

```
peek()
  if front == -1 OR front > rear
    print "Queue Underflow"
  else
    return queue[front]
```

<u>Algorithm for isEmpty( ) operation</u>

```
isEmpty()
  if front == -1 OR front > rear
    return 1     // queue is empty
  else
    return 0     // queue is not empty
```

Algorithm for isFull() operation

```
isFull()

  if rear == MAX_SIZE - 1
    return 1     // queue is full
  else
    return 0     // queue is not full
```

Types of Queue

There are four types of queues:

- Simple Queue
- Circular Queue
- Priority Queue
- Double-ended Queue

1. **Linear Queue**

A linear queue is a type of queue in which elements are inserted at the rear and deleted from the front.

It follows the FIFO (First-In, First-Out) principle, meaning the element inserted first is removed first.

However, in array implementation, it suffers from space wastage, because once elements are removed, the empty spaces at the front cannot be reused.

Examples: printer queue, message queue.

2. **Circular Queue**

A circular queue is an improved version of the linear queue in which the last position is connected back to the first position, forming a circle.

This allows the queue to reuse empty spaces, making better use of memory and improving efficiency.

It is especially useful when continuous insertion and deletion operations are performed.

Examples: CPU scheduling, memory management.

3. Priority Queue

A priority queue is a special type of queue in which each element is assigned a priority.

Elements are processed based on their priority rather than their insertion order.

Higher priority elements are removed first

If priorities are equal, FIFO is followed

Examples: operating system task scheduling, network packet scheduling.

Program to implement Linear Queue

```
#include <stdio.h>
#define MAX_SIZE 7
int queue[MAX_SIZE];
int front = -1, rear = -1;
void enqueue(int item);
void dequeue();
void peek();
void display();
int main() {
   int ch, x;
   while (1) {
      printf("\n1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &ch);
      if (ch == 1) {
         printf("Enter element to enqueue: ");
         scanf("%d", &x);
         enqueue(x);
      }
      else if (ch == 2)
         dequeue();
      else if (ch == 3)
         peek();
      else if (ch == 4)
         display();
      else if (ch == 5) {
         printf("Exited.\n");
         break;
      }
      else
         printf("Invalid choice!\n");
   }
   return 0;
}
```

```
void enqueue(int item) {
   if (rear == MAX_SIZE - 1)
      printf("<--- Queue Overflow --->\n");
   else {
      if (front == -1) front = 0;  // first element
      queue[++rear] = item;
      printf("Enqueued element: %d\n", item);
   }
}
void dequeue() {
   if (front == -1 || front > rear)
      printf("<--- Queue Underflow --->\n");
   else {
      printf("Dequeued element: %d\n", queue[front]);
      front++;
   }
}
void peek() {
   if (front == -1 || front > rear)
      printf("<--- Queue Underflow --->\n");
   else
      printf("Front element: %d\n", queue[front]);
}
void display() {
   if (front == -1 || front > rear)
      printf("<--- Queue Underflow --->\n");
   else {
      printf("Queue elements:\n");
      for (int i = front; i <= rear; i++)
         printf("%d\n", queue[i]);
   }
}
```

<u>Program to implement Circular Queue</u>

```
#include <stdio.h>
#define MAX_SIZE 5
int queue[MAX_SIZE];
int front = -1, rear = -1;
```

```
void enqueue(int item);
void dequeue();
void peek();
void display();
int main() {
   int ch, x;
   while(1) {
      printf("\n1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &ch);
      if(ch == 1) {
         printf("Enter element to enqueue: ");
         scanf("%d", &x);
         enqueue(x);
      }
      else if(ch == 2)
         dequeue();
      else if(ch == 3)
         peek();
      else if(ch == 4)
         display();
      else if(ch == 5) {
         printf("Exited.\n");
         break;
      }
      else
         printf("Invalid choice!\n");
   }
   return 0;
}
int isFull() {
   return ((front == 0 && rear == MAX_SIZE-1) || (rear + 1) % MAX_SIZE ==
front);
}
int isEmpty() {
   return (front == -1);
}
void enqueue(int item) {
   if(isFull())
```

```
        printf("<--- Circular Queue Overflow --->\n");
    else {
        if(front == -1) front = 0;
        rear = (rear + 1) % MAX_SIZE;
        queue[rear] = item;
        printf("Enqueued element: %d\n", item);
    }
}
void dequeue() {
    if(isEmpty())
        printf("<--- Circular Queue Underflow --->\n");
    else {
        printf("Dequeued element: %d\n", queue[front]);
        if(front == rear)  // last element removed
            front = rear = -1;
        else
            front = (front + 1) % MAX_SIZE;
    }
}
void peek() {
    if(isEmpty())
        printf("<--- Circular Queue Underflow --->\n");
    else
        printf("Front element: %d\n", queue[front]);
}
void display() {
    if(isEmpty()) {
        printf("<--- Circular Queue Underflow --->\n");
        return;
    }
    printf("Queue elements:\n");
    int i = front;
    while(1) {
        printf("%d\n", queue[i]);
        if(i == rear) break;
        i = (i + 1) % MAX_SIZE;
    }
}
```

Program to implement Circular Queue

```
#include <stdio.h>
#define MAX_SIZE 5
int queue[MAX_SIZE];
int priority[MAX_SIZE];
int n = 0;  // number of elements
void enqueue(int item, int pr);
void dequeue();
void peek();
void display();
int main() {
   int ch, x, pr;
   while(1) {
      printf("\n1. Enqueue\n2. Dequeue\n3. Peek\n4. Display\n5. Exit\n");
      printf("Enter your choice: ");
      scanf("%d", &ch);
      if(ch == 1) {
         printf("Enter element: ");
         scanf("%d", &x);
         printf("Enter priority: ");
         scanf("%d", &pr);
         enqueue(x, pr);
      }
      else if(ch == 2)
         dequeue();
      else if(ch == 3)
         peek();
      else if(ch == 4)
         display();
      else if(ch == 5) {
         printf("Exited.\n");
         break;
      }
      else
         printf("Invalid choice!\n");
   }
   return 0;
}

void enqueue(int item, int pr) {
```

```
    if(n == MAX_SIZE) {
        printf("<--- Priority Queue Overflow --->\n");
        return;
    }
    int i = n - 1;
    while(i >= 0 && priority[i] < pr) {  // higher priority first
        queue[i+1] = queue[i];
        priority[i+1] = priority[i];
        i--;
    }
    queue[i+1] = item;
    priority[i+1] = pr;
    n++;
    printf("Enqueued element: %d with priority %d\n", item, pr);
}
void dequeue() {
    if(n == 0) {
        printf("<--- Priority Queue Underflow --->\n");
        return;
    }
    printf("Dequeued element: %d with priority %d\n", queue[0], priority[0]);
    for(int i = 1; i < n; i++) {
        queue[i-1] = queue[i];
        priority[i-1] = priority[i];
    }
    n--;
}
void peek() {
    if(n == 0)
        printf("<--- Priority Queue Underflow --->\n");
    else
        printf("Front element: %d with priority %d\n", queue[0], priority[0]);
}
void display() {
    if(n == 0) {
        printf("<--- Priority Queue Underflow --->\n");
        return;
    }
    printf("Queue elements with priorities:\n");
    for(int i = 0; i < n; i++)
```

```
    printf("%d (priority %d)\n", queue[i], priority[i]);
}
```

Q.N. Difference between Stack, Queue and Linked List

Q.N. Difference between Linear Queue and Circular Queue

Time Complexity Chart

Queue Type	enqueue( )	dequeue( )	peek( )	isEmpty( )	isFull( )	display( )
Linear Queue	O(1)	O(1)	O(1)	O(1)	O(1)	O(n)
Circular Queue	O(1)	O(1)	O(1)	O(1)	O(1)	O(n)
Priority Queue	O(n)	O(1)	O(1)	O(1)	O(1)	O(n)